CMPT 225 — Assignment 1

Total: 81 marks · 9 parts (A–I)

Student Name: _____

Student ID: _____

Rules

- Code may be written in Java or in clear Java-like pseudocode.
- Use only the library classes a question explicitly allows.
- Show your work.

Conventions

Unless a question states otherwise, the following conventions hold.

- n is the input size.
- log means $\log_2$.
- "Running time" means worst-case running time.
- "Space" means auxiliary space: memory used beyond the input, including the function call stack.
- "Tightest bound" means give a Θ bound.
- Draw linked structures as boxes and arrows.
- Represent a `null` reference as $\varnothing$.

Grading

Part	Topic	Marks	Score
A	Asymptotic notation: concepts	24	
B	Asymptotic notation: proofs	REMOVED	
C	Analyzing time and space	15	
D	Singly linked lists	18	
E	Doubly linked lists	18	
F	Stacks	REMOVED	
G	Queues	6	
H	The function call stack	REMOVED	
I	Design problems	REMOVED	
	Total	**81**	

Part A — Asymptotic Notation: Concepts (24 marks)

A1. Definitions (6 marks)

Let $f(n)$ and $g(n)$ be functions from the positive integers to the non-negative real numbers.

(a) State the formal definition of $f(n) \in O(g(n))$. [/ 2]

(b) State the formal definition of $f(n) \in \Omega(g(n))$. [/ 2]

(c) State the formal definition of $f(n) \in \Theta(g(n))$. In one sentence, explain what a Θ bound tells you that an O bound alone does not. [/ 2]

A2. True or false? (12 marks)

For each statement, write **T** or **F** and justify it in one or two lines (a short calculation, a pair of constants, or a counterexample). Each part is worth 1.5 marks: 0.5 for the verdict and 1 for the justification.

(a) $3n^2 + 100n + 7 \in O(n^2)$ [/ 1.5]

(b) $n^2 \in O(n \log n)$ [/ 1.5]

(c) $2^{n+1} \in \Theta(2^n)$ [/ 1.5]

(d) $2^{2n} \in O(2^n)$ [/ 1.5]

(e) $\log_2 n \in \Theta(\log_{10} n)$ [/ 1.5]

(f) $(\log n)^3 \in O(n)$ [/ 1.5]

(g) If $f(n) \in O(g(n))$, then $g(n) \in \Omega(f(n))$. [/ 1.5]

(h) If an algorithm's worst-case running time is $\Theta(n^2)$, then it takes $\Omega(n^2)$ time on *every* input of size n. [/ 1.5]

A3. Ranking growth rates (6 marks)

Rank the twelve functions below in order of increasing asymptotic growth rate, assigning rank 1 to the slowest-growing function or group of functions. Functions that are Θ of each other must receive the same rank and be listed in the same row, separated by commas. Assign consecutive ranks to distinct growth classes, with no gaps between ranks. [/ 6]

n^2, $4^{n/2}$, $n \log n$, 100, $\sqrt{n}$, $n!$, $\log(n^2)$, $n^3/1000$, 2^n, $\log n$, n, $2^{\log n}$

Rank	**Function(s)**
1 (slowest)	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	

Part C — Analyzing Time and Space (15 marks)

C1. Code fragments (12 marks)

For each fragment, give the tightest bound on (i) the running time and (ii) the auxiliary space, both as functions of n. Justify the running time by showing the sum you used. Assume $n \geq 1$ and that `count` is a `static int` field. Each fragment is worth 3 marks: 2 for the running time with its justification and 1 for the space.

(a) [/3]

```
for (int i = 0; i < n; i++)
    for (int j = i; j < n; j++)
        count++;
```

(b) [/3]

```
for (int i = 1; i <= n; i *= 2)
    for (int j = 0; j < i; j++)
        count++;
```

(c) [/3]

```
for (int i = 0; i < n; i++)
    for (int j = 1; j < n; j *= 2)
        count++;
```

(d) [/3]

```
for (int i = n; i > 0; i /= 2)
    for (int j = 0; j < i; j++)
        count++;
```

C2. Predicting running time (3 marks)

A program takes **2 seconds** on an input of size $n = 1000$. Assume its running time is exactly proportional to the function given (constants and lower-order terms are negligible). Estimate its running time on an input of size $\boldsymbol{n = 4000}$ if the running time is proportional to:

(a) $n \log n$ [/ 1]

(b) n^2 [/ 1]

(c) 2^n [/ 1]

Hint: $\log 1000 \approx 10$. You may leave answer (c) as a power of 2, but say in words what it means in practice.

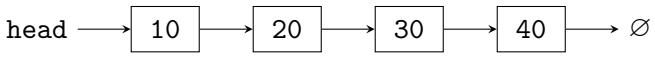
Part D — Singly Linked Lists (18 marks)

Use the `Node` class from the Appendix:

```
class Node {
    int  data;
    Node next;
    Node(int data)            { this(data, null); }
    Node(int data, Node next) { this.data = data; this.next = next; }
}
```

D1. Tracing references (6 marks)

The list starts as:



Then this code runs:

```
Node p = head.next;
Node q = p.next;
p.next = q.next;
q.next = head;
head = q;
p = new Node(25, p.next);
head.next.next.next = p;
```

(a) Draw the final list, starting at `head`, as boxes and arrows. [/ 3]

(b) Which node does `p` refer to at the end? Which node does `q` refer to? [/ 1]

(c) What does a loop that prints every value from `head` to the end of the list output? [/ 1]

(d) Is any node unreachable (and therefore eligible for garbage collection) when the code finishes? Explain briefly. [/ 1]

D2. Cost of list operations (5 marks)

(a) Fill in the worst-case running time (Θ) of each operation on a singly linked list holding n elements. "Head only" keeps just a `head` reference; "Head + tail" keeps both `head` and `tail` references. Neither version stores a size counter. (4 marks: 0.5 per cell, scored by row.)

Operation	**Head only**	**Head + tail**	**Score**
Insert at front			[/ 1]
Insert at back			[/ 1]
Remove from front			[/ 1]
Remove from back			[/ 1]

(b) In one sentence, explain your entry for "Remove from back" in the "Head + tail" column. [/ 1]

D3. Reversing a list (7 marks)

(a) Write `static Node reverse(Node head)`, which reverses a singly linked list **in place** (by relinking nodes, not by copying data or creating new nodes) and returns the new head. It must run in $\Theta(n)$ time, use $\Theta(1)$ auxiliary space, and work for empty and one-node lists.

[/ 4]

(b) A classmate wrote the recursive version below. What is its auxiliary space complexity, and why does it differ from your answer to (a)? [/ 2]

```
static Node reverseRec(Node head) {
    if (head == null || head.next == null) return head;
    Node rest = reverseRec(head.next);
    head.next.next = head;
    head.next = null;
    return rest;
}
```

(c) In `reverseRec`, which node does `head.next` refer to when the line `head.next.next = head;` runs, and why does that line put `head` in the right place? [/ 1]

Part E — Doubly Linked Lists (18 marks)

Use the `DNode` class from the Appendix:

```
class DNode {
    int   data;
    DNode prev, next;
    DNode(int data) { this(data, null, null); }
    DNode(int data, DNode prev, DNode next) {
        this.data = data; this.prev = prev; this.next = next;
    }
}
```

The list uses two **sentinel** (dummy) nodes, `header` and `trailer`, which hold no data. The first real element is `header.next` and the last is `trailer.prev`.

E1. Sentinels (4 marks)

(a) Draw the empty list, showing all four reference fields (`prev` and `next`) of `header` and `trailer`. [/ 1]

(b) Draw the list holding A, B, C (in that order), showing every `prev` and `next` reference. [/ 1]

(c) Give two concrete advantages of sentinel nodes. Each advantage must refer to a specific operation. [/ 2]

E2. $\Theta(1)$ insertion and removal (8 marks)

(a) Write `void insertAfter(DNode p, int x)`, which inserts a new node holding `x` immediately after node `p`. `p` may be `header` but is never `trailer`. It must run in $\Theta(1)$ time. [/ 4]

(b) Write `int remove(DNode p)`, which unlinks node `p` (never a sentinel) from the list and returns the value it held. It must run in $\Theta(1)$ time. [/ 3]

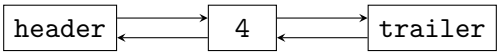
(c) Using only `insertAfter`, write one-line bodies for `pushFront(int x)` and `pushBack(int x)`. [/ 1]

E3. Find the bug (6 marks)

A student wrote:

```
void insertAfterBuggy(DNode p, int x) {
    DNode q = new DNode(x);
    p.next = q;          // line 1
    q.prev = p;          // line 2
    q.next = p.next;     // line 3
    p.next.prev = q;     // line 4
}
```

The list is shown below (upper arrows are `next` references, lower arrows are `prev` references). The student calls `insertAfterBuggy(p, 7)`, where `p` refers to the node holding 4.



(a) Draw the structure after the call. Show both references of the node holding 4, of the new node, and of `trailer`. [/ 2]

(b) What happens when the list is printed by following `next` references from `header`? What happens when it is printed backwards by following `prev` references from `trailer`? [/ 2]

(c) Rewrite the method so that it is correct. [/ 2]

Part G — Queues (6 marks)

G1. A linked queue (6 marks)

Implement a queue of integers with a singly linked list of `Node`s (see the Appendix), keeping both a `head` and a `tail` reference so that `enqueue` and `dequeue` both run in $\Theta(1)$ time.

```
public class LinkedQueue {
    private Node head = null;
    private Node tail = null;

    public void enqueue(int x) { /* your code */ }

    // Throws NoSuchElementException if the queue is empty.
    public int dequeue()       { /* your code */ }

    public boolean isEmpty()   { return head == null; }
}
```

(a) Which end of the list must be the **front** of the queue (where `dequeue` happens)? Explain why the other choice cannot give $\Theta(1)$ for both operations. [/ 1]

(b) Write `enqueue` and `dequeue`. Pay particular attention to enqueuing into an empty queue and to dequeuing the last element. [/ 5]

Appendix — Reference Sheet

Node classes

```
class Node {                      // singly linked
    int  data;
    Node next;
    Node(int data)            { this(data, null); }
    Node(int data, Node next) { this.data = data; this.next = next; }
}

class DNode {                     // doubly linked
    int   data;
    DNode prev, next;
    DNode(int data) { this(data, null, null); }
    DNode(int data, DNode prev, DNode next) {
        this.data = data; this.prev = prev; this.next = next;
    }
}
```

Assume `import java.util.*;` at the top of every file.

Java library interfaces

Stack: `Deque<T> s = new ArrayDeque<>();`		**Queue:** `Queue<T> q = new ArrayDeque<>();`	
`s.push(x)`	add `x` on top	`q.offer(x)`	add `x` at the back
`s.pop()`	remove and return the top	`q.poll()`	remove and return the front (`null` if empty)
`s.peek()`	return the top without removing it	`q.peek()`	return the front without removing it
`s.isEmpty()`	`true` if empty	`q.isEmpty()`	`true` if empty
`s.size()`	number of elements	`q.size()`	number of elements

- `java.util.Stack<T>` offers the same five stack methods. Calling `pop()` on an empty stack throws an exception.
- Collections store objects, so `int` values are automatically boxed to `Integer` and unboxed back.
- Each operation above runs in $\Theta(1)$ time (amortized $\Theta(1)$ for `push` and `offer`).